

Guide de lecture : Every Patch Is a Guess — G-EPI-000

Thèse, hypothèses, familles et limites

Ted “Tedsig42” Kouhouenou ted.sig42@tutamail.com

Summary: Guide autonome pour lire, expliquer et défendre le papier. Il donne la thèse, les trois familles, le modèle de mutation, la boucle d’audit, les hypothèses cas par cas, les limites et une carte exhaustive des exemples. C’est le document de référence à lire avant les modules.

Prerequisites : Aucun prérequis technique ; lire les modules avec ce guide ouvert

Version: 1.0

Contents

I	**La thèse en une page**	**2**
II	**Comment l'auteur construit sa preuve**	**4**
III	**Les trois familles**	**6**
IV	**Le modèle de mutation**	**8**
V	**La boucle d'audit différentiel**	**10**
VI	**Boîte à outils minimale**	**12**
VII	**Les hypothèses, cas par cas**	**14**
VIII	**Les motifs qui reviennent**	**18**
IX	**Les limites du papier**	**19**
X	**Comment le présenter en vingt minutes**	**20**
XI	**Carte exhaustive des exemples**	**21**
XII	**Sources et liens**	**23**

Chapter I

La thèse en une page

Le papier ne dit pas que les correctifs sont mauvais. Il dit :

> Un correctif de sécurité est une hypothèse sur ce qui pouvait mal tourner. Cette hypothèse est locale. Le système, lui, ne l'est pas.

Un développeur corrige le chemin qu'il a observé. Il ajoute un test, un verrou, une validation de type ou une remise à zéro. Ce faisant, il affirme implicitement :

- « le bug est dans cette fonction » ;
- « ce chemin est le seul atteignable » ;
- « cette propriété suffit à protéger la ressource » ;
- « les autres appelants sont déjà protégés » ;
- « la même règle vaut pour toutes les branches, tous les opcodes, tous les protocoles ou tous les backends ».

Le papier prend chacune de ces phrases au sérieux, puis cherche le contre-exemple. La faille n'est pas dans la ligne ajoutée : elle est dans l'écart entre le modèle du correctif et le modèle réel du système.

D'où la formule centrale :

> **Every patch encodes an assumption about what can go wrong. Your job is to find the assumption — and watch what it mutates into.**

La méthode qui en découle :

1. classer le composant dans une famille architecturale ;
2. extraire l'hypothèse du diff ;
3. chercher les chemins structurellement équivalents qui ne reçoivent pas la même vérification ;
4. trier les candidats sans les confondre avec des exploits ;

5. mesurer le temps pendant lequel l'hypothèse reste fausse dans le parc réel : le *patch gap*.

Chapter II

Comment l'auteur construit sa preuve

Le papier mélange quatre niveaux de preuve. Il faut les distinguer pour le présenter honnêtement.

Niveau	**Ce que c'est**	**Exemples**
Démontré	Étude de cas détaillée, avec code, commit, hypothèse et chemin de mutation	nf_tables, eBPF, io_uring, Windows
Observé	Motif répété dans le corpus de 62 CVE, sans analyse exhaustive	classifieur, dataset, statistiques
Cartographié	Cas relié au framework, détails dans l'archive ou les sources externes	FreeBSD, XNU
Extrapolé	Affirmation de portée générale à partir d'un échantillon	« les trois familles couvrent tout système stateful »

La force du papier est de ne pas cacher le troisième et le quatrième niveau. La section 18.11 reconnaît explicitement les faux positifs, les faux négatifs, le chevauchement des familles et la portée principalement Linux.

Le corpus

Le dépôt compagnon contient `validation/dataset.csv` (62 CVE) et `validation/methodology.md`. Le classifieur transforme des signaux structurels en famille. Le risque évident est circulaire : si les CVE ont été choisies parce qu'elles ressemblent aux familles, le classifieur confirme le choix. Le papier le reconnaît partiellement ; un auditeur doit le dire plus fort.

Les études de cas

Quatre domaines servent de preuve :

- nf_tables : dispersion architecturale et correctif incomplet ;
- eBPF/JIT : oracle prédictif et pruning ;
- io_uring : proxy concurrent et durée de vie ;
- Windows : machine à états d'authentification et frontières.

Node.js, Apache MINA et FreeBSD/XNU servent de warm-ups ou de mappings cross-domain. Ils ne remplacent pas les études de cas complètes.

Chapter III

Les trois familles

Ce ne sont pas des catégories de gravité. Ce sont des *formes de gestion* qui prédisent la manière dont un correctif va échouer.

Famille	**Question diagnostique**	**Mode d'échec**	**Exemples**
Logical Engine	Le composant gère-t-il des objets avec un cycle de vie explicite : create, destroy, abort, rollback ?	Une transition d'état n'a pas les mêmes invariants que le chemin nominal.	nf_tables, MINA, authentification Windows, COM
Predictive Oracle	Le composant analyse-t-il une entrée avant de l'exécuter ?	Le modèle de l'analyseur est incomplet par rapport au backend réel.	vérificateur eBPF, JIT, `path.normalize()`
Concurrent Proxy	Le composant partage-t-il de la mémoire ou des ressources avec une synchronisation minimale ?	Le moment de la validation n'est pas le moment de l'utilisation.	io_uring, rings, files, completions

Nuance importante sur « Logical Engine »

Le nom peut tromper. Un Logical Engine n'est pas « un bug de logique » au sens vague. C'est un composant qui gère des *cycles de vie* et des *transitions d'état*. Le papier insiste sur l'asymétrie :

- le chemin nominal fait `allocate -> initialize -> commit` ;
- le chemin d'erreur fait `allocate -> abort` ;
- le chemin de nettoyage fait `destroy` sans repasser par les mêmes gardes ;
- le correctif ajoute une garde sur un chemin, pas sur les autres.

C'est pour cela que les questions les plus productives sont souvent : « que fait le chemin d'abandon ? », « que fait le rollback ? », « que fait le timeout ? ».

Pourquoi les familles se chevauchent

Un même composant peut être :

- Predictive Oracle au sommet de la pile (il vérifie une entrée) ;
- Logical Engine trois frames plus bas (il gère un état) ;
- Concurrent Proxy sur le côté (il partage une ressource).

Le papier le dit dans la section 14.1 : le vérificateur eBPF est un oracle, mais ses décisions de pruning sont aussi un moteur d'état, et les maps partagées ajoutent un risque concurrent. La classification n'est donc pas une partition formelle : c'est une *heuristique de recherche*.

Chapter IV

Le modèle de mutation

Incomplete-fix variant

Le correctif ferme le chemin observé, mais la même faiblesse existe déjà ailleurs, souvent dans une fonction voisine ou une branche voisine.

- nf_tables : `CVE-2023-4244` retire une garde ; `CVE-2024-1085` apparaît plus tard dans le même sous-système.
- MINA : `CVE-2026-41409` est corrigée sur 2.0.x, mais 2.1.x et 2.2.x reçoivent `CVE-2026-42778`.
- Node.js : le correctif des noms de lecteur laisse passer les noms de périphériques, puis les chemins UNC.

Architectural scatter

Le correctif ferme une instanciation d'une classe, mais l'architecture contient plusieurs chemins indépendants qui partagent la même hypothèse.

- nf_tables : `CVE-2023-35001` (byteorder) et `CVE-2024-1086` (verdict) sont deux collisions d'une même dispersion.
- eBPF : `CVE-2020-27194` corrige `or`, mais `and` et `xor` restent vulnérables ; `CVE-2021-3490` couvre les trois.
- io_uring : `CVE-2021-20226` (refcount de fichiers) et `CVE-2021-3491` (longueur de buffer) sont deux chemins du même ring.
- Windows : cinq CVE, cinq protocoles, une seule racine de réflexion.

Structural symmetry bypass

Le correctif ajoute une garde dans une fonction, mais une fonction structurellement symétrique ne la reçoit pas. C'est un cas particulier d'architectural scatter, très fréquent dans eBPF et io_uring.

Le patch gap

Le patch gap n'est pas seulement « commit amont vers stable ». C'est la chaîne complète :

```
vulnerabilite connue
  -> correctif ecrit
  -> correctif livre
  -> correctif active
  -> correctif reellement present dans le parc
```

Chaque étape peut ajouter du temps. Le papier utilise OtterRoot et V8 mobile pour montrer que la fenêtre d'exposition peut être mesurée en semaines ou en mois, même après publication du correctif.

La fenêtre de mutation

Le papier observe souvent une fenêtre de 12 à 18 mois entre un correctif et la variante suivante. Ce n'est pas une loi physique. C'est une observation de corpus, utile pour prioriser :

- un sous-système qui a eu une CVE récente ;
- un correctif récent ;
- un twin dans un autre fichier ;
- un chemin atteignable sans privilège.

Chapter V

La boucle d'audit différentiel

La section 4 donne trois étapes ; la section 18.3 donne les commandes complètes.

Étape 1 — Identifier le code ajouté

Chaque ligne ajoutée encode une hypothèse :

```
+ if (condition) return -EINVAL;
  -> existe-t-il un autre point d'entree sans ce test ?

+ mutex_lock(&x->lock);
  -> existe-t-il un autre chemin qui touche x sans ce verrou ?

+ if (!nft_is_active_next(net, x)) continue;
  -> existe-t-il une fonction qui utilise encore l'ancienne generation ?
```

Commandes utiles :

```
git show <fix_commit> > fix.patch
grep '^+' fix.patch | grep -v '^+++' | head -40
grep '^+' fix.patch | grep 'struct\|->use\|->refcount\|genmask'
```

Étape 2 — Trouver le code parallèle non modifié

Chercher les fonctions qui opèrent sur la même structure et font un travail analogue :

```
grep -RIn --include='*.c' 'nft_setelem' net/netfilter/
grep -RIn --include='*.c' 'nft_is_active_next' net/netfilter/
```

`ctags` et Weggli sont optionnels. La commande `grep -RIn` donne déjà le fichier, la ligne et le texte ; c'est suffisant pour formuler et vérifier une hypothèse.

Étape 3 — Vérifier la symétrie

La ressemblance ne suffit pas. Il faut montrer que le twin partage réellement l'invariant, qu'il n'est pas protégé par un autre mécanisme, et qu'il est atteignable.

Questions de la section 18.7 :

1. Quelle famille ?

2. Quel est le chemin nominal, quels sont les chemins exceptionnels ?

3. Que suppose chaque fonction sur l'état qu'elle reçoit ?

4. Le correctif est-il symétrique ?

5. Qu'est-ce qui a changé récemment pour la performance ?

La question 4 est présentée comme la plus productive. C'est elle qui mène à `CVE-2024-1085` et `CVE-2024-0582`.

Chapter VI

Boîte à outils minimale

Cette piscine ne demande pas de connaître Git en profondeur. Savoir `commit` et `push` suffit pour rendre un exercice. Les commandes ci-dessous sont en lecture seule : elles ne modifient pas le dépôt.

Git : lire l'historique

```
git log --oneline -5
git show COMMIT
git show COMMIT -- chemin/du/fichier.c
git diff
git log -p -- chemin/du/fichier.c
```

- `git log -oneline` affiche les derniers commits, un par ligne.
- `git show COMMIT` affiche le diff d'un commit.
- `git diff` affiche ce qui n'est pas encore commité.
- `git log -p` affiche l'historique avec les diffs.

Aucun `rebase`, `cherry-pick` ou `force-push` n'est nécessaire. Si vous ne connaissez pas une commande, notez celle que vous avez utilisée et son résultat : c'est ce que le correcteur veut voir.

grep : chercher sans ctags

`ctags` n'est pas requis. Pour trouver une fonction, un appel ou une structure :

```
grep -RIn --include='*.c' 'nft_setelem' net/netfilter/
grep -n 'nft_is_active_next' net/netfilter/nf_tables_api.c
grep -RIn '->use' net/netfilter/
```

- `-R` descend dans les sous-dossiers ;
- `-I` ignore les fichiers binaires ;
- `-n` affiche le numéro de ligne ;

- `--include='*.c'` limite aux sources C.

Pour trouver les définitions plutôt que les appels :

```
grep -RIn --include='*.c' '^[a-zA-Z_].*nft_setelem.*(' net/netfilter/
```

ctags : seulement si disponible

Si `ctags` est installé, il peut accélérer la recherche :

```
ctags -R --c-kinds=f --fields=+n .
grep 'nft_setelem' tags
```

Mais un rendu qui utilise uniquement `grep` et `find` est parfaitement valide. Le critère n'est pas l'outil : c'est la commande reproductible et l'hypothèse qu'elle teste.

Chapter VII

Les hypothèses, cas par cas

nf_tables

Cas	Hypothèse du correctif	Angle mort
`CVE-2023-35001`	`priv->len` octets depuis le registre de base suffisent à borner l'accès.	La taille de l'union et le stride de 4 ne correspondent pas à `len/2`.
`CVE-2024-1086`	La correction du verdict `NF_DROP` ferme la classe.	Un autre chemin de verdict produit encore un double-free.
`CVE-2023-4244`	Retirer la marque « busy » et l'API de batch GC simplifie le cycle de vie.	Une garde préexistante masquait une faiblesse ; `CVE-2024-1085` la révèle.
`CVE-2024-1085`	Vérifier la génération « next » avant `kfree()` suffit.	Le bug est une transition de génération, pas un simple test manquant.
`CVE-2024-26809`	Le commit de nettoyage pipapo couvre les éléments déjà commités.	`priv->dirty` reste vrai si `set->dead` saute `->commit()` ; double-free.

Node.js

`path.normalize()` est un Predictive Oracle : il prédit qu'un chemin est sûr pour le système de fichiers.

Cas	Hypothèse	Angle mort
`CVE-2025-23084`	Un chemin sans lettre de lecteur au début est relatif.	`C:..\sensitive` est relatif au répertoire courant du lecteur C:.
`CVE-2025-27210`	Filtrer les noms de lecteur suffit.	`CON`, `PRN`, `AUX`, `NUL`, `COM1-9`, `LPT1-9` sont des périphériques.
UNC non assigné	Filtrer les périphériques suffit.	Les chemins UNC restent valides et peuvent sortir du sous-arbre.

La leçon : chaque correctif ajoute une règle au modèle. Il ne prouve pas que le modèle couvre toute la sémantique Windows.

Apache MINA

Le désérialiseur Java est un Logical Engine :

```
allocate -> initialize -> resolve -> check
```

Cas	Hypothèse	Angle mort
CVE-2024-52046	Aucun filtre n'est nécessaire si le flux est de confiance.	Instanciation arbitraire avant tout contrôle.
CVE-2026-41409	Une allowlist vérifiée dans `getObject()` protège l'instanciation.	Les initialiseurs statiques s'exécutent avant la garde.
CVE-2026-42778	Le correctif de 2.0.x est complet pour toutes les branches.	2.1.x et 2.2.x ont besoin d'une seconde correction.

La question de chasse est : « la correction est-elle appliquée à toutes les transitions d'état structurellement équivalentes, ou seulement à celle où le bug a été observé ? »

eBPF/JIT

Le vérificateur est un Predictive Oracle. Son modèle de domaine est une sur-approximation censée être sound ; chaque cas manqué crée un chemin concret que le modèle n'a pas anticipé.

Cas	Hypothèse	Angle mort
CVE-2020-27194	Les mêmes champs de bornes conviennent aux opérations 32 et 64 bits.	`scalar32_min_max_or()` utilise des champs 64 bits.
CVE-2021-3490	Le correctif de `or` couvre toutes les opérations ALU32.	`and`, `xor` et le cas const-subreg ne sont pas corrigés.
CVE-2022-23222	Faire de l'arithmétique sur un pointeur nullable produit un pointeur valide.	`PTR_TO_ALLOC_MEM_OR_NULL` n'est pas rejeté avant le null-check.
CVE-2023-2163	Si un registre est `!precise`, ses bornes n'importent pas pour le pruning.	La précision n'est pas propagée à travers les sauts conditionnels.

Le cas CVE-2023-2163 est important : le raccourci `!precise` existe encore aujourd'hui. Le correctif n'est pas de supprimer le raccourci, mais de propager la précision dans `backtrack_insn()`.

io_uring

io_uring est un Concurrent Proxy : les ring buffers sont partagés entre utilisateur et noyau, et chaque champ d'un SQE lu avec `READ_ONCE()` est une fenêtre TOCTOU potentielle.

Cas	Hypothèse	Angle mort
`CVE-2021-20226`	Corriger le refcount de `files_struct` dans `io_grab_files()` ferme la classe.	Un autre chemin du même ring ne valide pas la durée de vie.
`CVE-2021-3491`	La validation des buffers fournis est suffisante.	La longueur n'est pas plafonnée à `MAX_RW_COUNT`.
`CVE-2023-2598`	Le fast path des pages composées est sûr.	Deux adresses virtuelles peuvent mapper la même page physique.
`CVE-2024-0582`	Le durcissement de l'enregistrement des buffers fixes couvre tous les buffers.	Le PBUF ring a son propre cycle de vie ; `mmap` survit à `unregister`.

La formule du papier : « When every path is fast, which one is safe ? »

Windows

La chaîne d'authentification Windows est un Logical Engine réparti sur plusieurs protocoles. Chaque correctif ajoute une garde à un composant sans établir l'invariant global.

Cas	Hypothèse	Angle mort
CVE-2025-33073	Bloquer le CMTI dans `mrxsmb.sys` ferme la réflexion.	D'autres primitives de coercition et de relais restent possibles.
CVE-2025-58726	Un SPN supprimé ne peut plus être utilisé.	Le compte de service existe encore ; le KDC émet un ticket pour l'ancien SPN.
CVE-2026-24294	Le contrôle loopback couvre les ports SMB standards.	Un serveur SMB local sur un port arbitraire peut recevoir et relayer le handshake.
CVE-2026-26128	La normalisation DNS et SPN est cohérente.	Le KDC normalise Unicode, DnsCache non ; un homoglyphe passe.
CVE-2026-26119	L'EPA est appliqué aux endpoints WAC.	La validation du contexte d'authentification est contournable.
CVE-2026-50343	Le problème est la carte de plugins `InstallService`.	La classe COM CrossDevice reste enregistrée sous `%PROGRAMDATA%`.
CVE-2026-66804	Le correctif de `InstallService` ferme la chaîne.	L'enregistrement COM orphelin reste une frontière de chargement.

FreeBSD et XNU

La section 18.11 cite :

- FreeBSD `CVE-2026-42511` (`dhclient`) ;
- FreeBSD `CVE-2026-4747` (NFS) ;
- XNU IOKit `IOUserClient` et Mach IPC.

Ces cas sont *cartographiés*, pas analysés en profondeur dans le corps du papier. Les détails sont annoncés dans `cross-kernel/` de l'archive. Il faut donc les traiter comme des hypothèses de portée, pas comme des preuves équivalentes aux études nf_tables/eBPF/io_uring.

Chapter VIII

Les motifs qui reviennent

Le papier ne donne pas une liste de bugs. Il donne des *patterns de dispersion* :

- **par branche** : 2.0.x corrigé, 2.1.x oublié (MINA) ;
- **par opcode** : un chemin io_uring corrigé, les autres non ;
- **par protocole** : SMB corrigé, HTTP/Kerberos non (Windows) ;
- **par type** : `or` corrigé, `and`/`xor` non (eBPF) ;
- **par génération** : `nft_is_active` au lieu de `nft_is_active_next` ;
- **par backend** : vérificateur corrigé, JIT non ;
- **par chemin de nettoyage** : commit corrigé, abort/rollback non.

La question d'audit se déplace donc de « où est le bug ? » vers « quelle dimension le correctif n'a-t-il pas couverte ? ».

Chapter IX

Les limites du papier

À assumer explicitement en soutenance :

1. **Faux positifs** : un twin structurel peut être inatteignable ou protégé par un autre invariant.

2. **Faux négatifs** : les interactions cross-family et les comportements matériels peuvent échapper à une classification mono-famille.

3. **Chevauchement** : les trois familles ne sont pas mutuellement exclusives ; la classification est une heuristique.

4. **Portée** : le framework est développé sur Linux, validé sur Windows, cartographié sur FreeBSD/XNU.

5. **Circularité possible** : le corpus et le classifieur peuvent se confirmer l'un l'autre.

6. **Candidat n'est pas exploit** : le framework produit des candidats, pas des vulnérabilités confirmées.

7. **Outils imparfaits** : les macros, `void *`, les unions, les appels indirects et la concurrence dégradent l'AST (section 18.1).

8. **Patch gap non mesuré partout** : les dates de commit ne disent pas quand un correctif est réellement activé dans le parc.

La concession la plus honnête : le papier prouve surtout que *la méthode trouve des cas réels dans les sous-systèmes qu'il a choisis*, pas qu'elle trouve tous les cas dans tous les systèmes.

Chapter X

Comment le présenter en vingt minutes

1. **3 min — le problème** : un patch est une hypothèse locale ; le système ne l'est pas.

2. **5 min — les trois familles** : Logical Engine, Predictive Oracle, Concurrent Proxy, avec un exemple minimal pour chacune.

3. **6 min — la boucle** : diff, twin, vérification ; montrer un cas nf_tables et un faux positif volontaire.

4. **3 min — les preuves** : corpus, études de cas, patch gap.

5. **3 min — les limites et ce que l'auditeur peut réutiliser demain**.

Phrase de conclusion possible :

> « Le correctif n'est pas la fin de l'analyse. C'est l'énoncé d'une hypothèse qu'il faut tester ailleurs. »

Chapter XI

Carte exhaustive des exemples

CVE	Section	Famille	Hypothèse / angle mort	Module
CVE-2023-35001	6.2	Predictive Oracle	`priv->len` borne l'accès ; la taille de l'union et le stride non.	M-EPI-003, A-EPI-002
CVE-2024-1086	6.2	Predictive Oracle	Le verdict `NF_DROP` est corrigé ; un autre chemin de verdict non.	M-EPI-003, A-EPI-002
CVE-2023-4244	6.3	Logical Engine	Retirer la garde simplifie le cycle de vie ; elle masquait une faiblesse.	M-EPI-000, M-EPI-003
CVE-2024-1085	6.3	Logical Engine	Vérifier la génération next suffit ; le bug est une transition d'état.	M-EPI-000, M-EPI-003
CVE-2025-23084	13.1	Predictive Oracle	Un chemin sans lecteur est relatif ; `C:..` ne l'est pas.	A-EPI-006
CVE-2025-27210	13.1	Predictive Oracle	Filtrer les lecteurs suffit ; les noms de périphériques passent.	A-EPI-006
UNC non assigné	13.1	Predictive Oracle	Filtrer les périphériques suffit ; les chemins UNC passent.	A-EPI-006
CVE-2024-52046	13.2	Logical Engine	Aucun filtre n'est nécessaire ; instanciation avant contrôle.	A-EPI-006
CVE-2026-41409	13.2	Logical Engine	L'allowlist protège ; les initialiseurs statiques s'exécutent avant.	A-EPI-006
CVE-2026-42778	13.2	Logical Engine	2.0.x est complet ; 2.1.x et 2.2.x ne le sont pas.	A-EPI-006
CVE-2020-27194	14.2	Predictive Oracle	Les bornes 64 bits conviennent au 32 bits ; non.	M-EPI-004, A-EPI-003
CVE-2021-3490	14.3	Predictive Oracle	`or` couvre tout l'ALU32 ; `and`/`xor` restent.	M-EPI-004, A-EPI-003
CVE-2022-23222	14.4	Predictive Oracle	L'arithmétique sur OR_NULL produit un pointeur valide ; non.	M-EPI-004, A-EPI-003
CVE-2023-2163	14.5	Predictive Oracle	`!precise` ignore les bornes ; la précision n'est pas propagée.	M-EPI-004, A-EPI-003

CVE	Section	Famille	Hypothèse / angle mort	Module
CVE-2021-20226	15.2	Concurrent Proxy	Le refcount de `files_struct` ferme la classe ; un autre chemin non.	M-EPI-004, A-EPI-003
CVE-2021-3491	15.2	Concurrent Proxy	La validation des buffers suffit ; la longueur n'est pas plafonnée.	M-EPI-004, A-EPI-003
CVE-2023-2598	15.3	Concurrent Proxy	Le fast path des pages composées est sûr ; deux mappings physiques non.	M-EPI-004, A-EPI-003
CVE-2024-0582	15.3	Concurrent Proxy	Le durcissement des buffers fixes couvre tout ; le PBUF ring non.	M-EPI-004, A-EPI-003
CVE-2025-33073	16.2	Logical Engine	Le CMTI ferme la réflexion ; d'autres coercitions restent.	A-EPI-004
CVE-2025-58726	16.2	Logical Engine	Un SPN supprimé est inutilisable ; le compte existe encore.	A-EPI-004
CVE-2026-24294	16.2	Logical Engine	Le loopback couvre les ports standards ; un port arbitraire passe.	A-EPI-004
CVE-2026-26128	16.2	Logical Engine	DNS et SPN normalisent pareil ; le KDC normalise Unicode.	A-EPI-004
CVE-2026-26119	16.2	Logical Engine	L'EPA protège WAC ; le contexte d'authentification est contournable.	A-EPI-004
CVE-2026-42511	18.11	Logical Engine ?	Mapping FreeBSD `dhclient`, détails dans l'archive.	A-EPI-006
CVE-2026-4747	18.11	Concurrent Proxy ?	Mapping FreeBSD NFS, détails dans l'archive.	A-EPI-006

Les CVE CVE-2024-26809, CVE-2026-50343 et CVE-2026-66804 sont des prolongements 2024-2026 utilisés dans les modules ; elles ne font pas partie des 24 CVE citées dans le corps du papier.

Chapter XII

Sources et liens

- Papier : `every-patch-is-a-guess-full.txt`.
- Dépôt compagnon : `rules/`, `validation/`, `classifier/`.
- Project Zero, *How to fix a bug in a fix* : https://projectzero.google/2026/10/emergency-patching.html
- Project Zero, *Dangling COM Object Registrations* : https://projectzero.google/2026/09/windows-dangling-com.html
- Project Zero, *Testing race conditions with memory access tracing and stack-based delay injection* : https://projectzero.google/2026/09/maccconc-race-condition.html
- OtterSec, *OtterRoot: Netfilter universal root 1-day* : https://osec.io/blog/netfilter-universal-root-1-day/
- OtterSec, *Patch gap to mobile renderer RCE* : https://osec.io/blog/patch-gap-to-mobile-renderer-rce/
- Calif, *Dark Elevator* : https://blog.calif.io/p/dark-elevator-windows-install-service
- Synacktiv, *The Gift That Keeps on Giving* : https://www.synacktiv.com/sites/default/files/2026-05/bypassing_authentication_reflections_mitigations_for_system_shells.pdf
- NVD : https://nvd.nist.gov/
- Node.js, *path* : https://nodejs.org/api/path.html
- Apache MINA, avis CVE-2024-52046, CVE-2026-41409, CVE-2026-42778.
- FreeBSD, avis de sécurité : https://www.freebsd.org/security/advisories/